Natural Language Processing

Prof. Ahmed Guessoum
The National Higher School of AI

Chapter 10

Large Language Models

Introduction to Large Language Models

LLMs

- In this chapter we formalize the idea of pretraining: learning knowledge about language and the world from vast amounts of text—and call the resulting pretrained language models **Large Language Models (LLMs)**.
- LLMs have been transformative, especially in tasks where text generation is required (MT, summarization, QAS, chatbots...)
- Text generation, code generation, and image generation together constitute the important new area of **generative AI**.
- Almost any NLP task can be modeled as word prediction in a large language model,

Language models

- Remember the simple n-gram language model
  - Assigns probabilities to sequences of words
  - Generates text by sampling possible next words
  - Is trained on counts computed from lots of text
- Large language models are similar and different:
  - Assign probabilities to sequences of words
  - Generate text by sampling possible next words
  - **Are trained by learning to guess the next word**

Large language models

- Even through pretrained only to predict words
- Learn a lot of useful language knowledge
- Since training on a **lot** of text

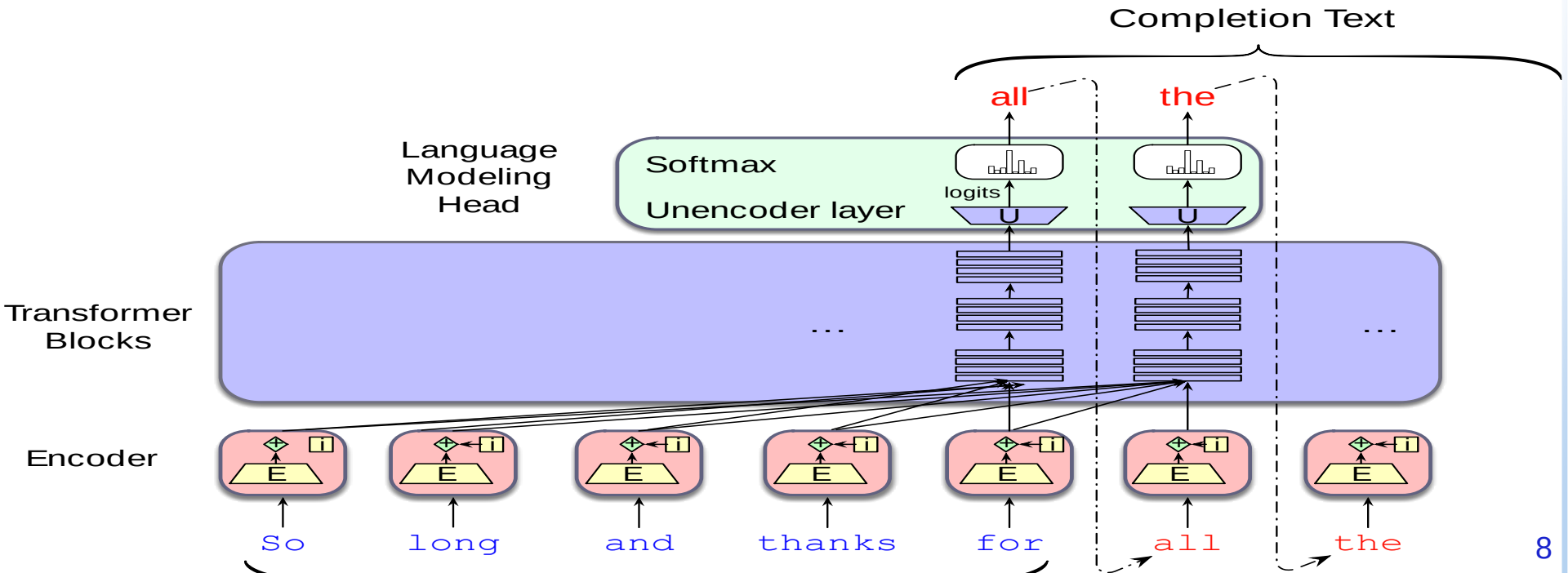
Large Language Models : What tasks can they do?

Big idea

- Many tasks can be turned into tasks of predicting words!

Conditional Generation

- **Conditional generation**: task of generating text conditioned on an input piece of text.
- The LLM takes an input piece of text (a prompt), and then continues generating text token by token, conditioned on the prompt.
- Transformers have long contexts (many thousands of tokens) which makes them very powerful for conditional generation, because they can look back very far into the prompting text.



Text Completion

- Example: simple task of text completion of "*So long and thanks for*"
- Here a LM is given a text prefix and is asked to generate a possible completion.
- During generation, the model has direct access to the priming context as well as to all of its own subsequently generated outputs (as fits in the large context window).
- Key to the **Power of LLMs built from transformers: t**his ability to incorporate
  - the entirety of the earlier context and
  - generated outputs at each time step.

Practical NLP tasks as word prediction!

- Sentiment analysis: “I like Jackie Chan”

1. We give the language model this string (prompt):

```
The sentiment of the sentence
"I like Jackie Chan" is:
```

1. And see what word it thinks comes next:

P(positive|The sentiment of the sentence ‘‘I like Jackie Chan" is:)

P(negative|The sentiment of the sentence ‘‘I like Jackie Chan" is:)

Framing tasks as conditional generation

QA: "Who wrote The Origin of Species" (Factual Q)

1. We give the language model this string:

   `Q: Who wrote the book "The Origin of Species"? A:`

   (token "A:" suggests that an answer should come next.)

2. and see what word it thinks comes next (i.e. highest probability):

   $P(w|$`Q: Who wrote the book "The Origin of Species"? A:`$)$

3. … and iterate:

$P(w|$`Q: Who wrote the book "The Origin of Species"? A: Charles`$)$

Text Summarization

Original

The only thing crazier than a guy in snowbound Massachusetts boxing up the powdery white stuff and offering it for sale online? People are actually buying it. For $89, self-styled entrepreneur Kyle Waring will ship you 6 pounds of Boston-area snow in an insulated Styrofoam box – enough for 10 to 15 snowballs, he says.

But not if you live in New England or surrounding states. "We will not ship snow to any states in the northeast!" says Waring's website, ShipSnowYo.com. "We're in the business of expunging snow!"

His website and social media accounts claim to have filled more than 133 orders for snow – more than 30 on Tuesday alone, his busiest day yet. With more than 45 total inches, Boston has set a record this winter for the snowiest month in its history. Most residents see the huge piles of snow choking their yards and sidewalks as a nuisance, but Waring saw an opportunity.

According to Boston.com, it all started a few weeks ago, when Waring and his wife were shoveling deep snow from their yard in Manchester-by-the-Sea, a coastal suburb north of Boston. He joked about shipping the stuff to friends and family in warmer states, and an idea was born. [...]

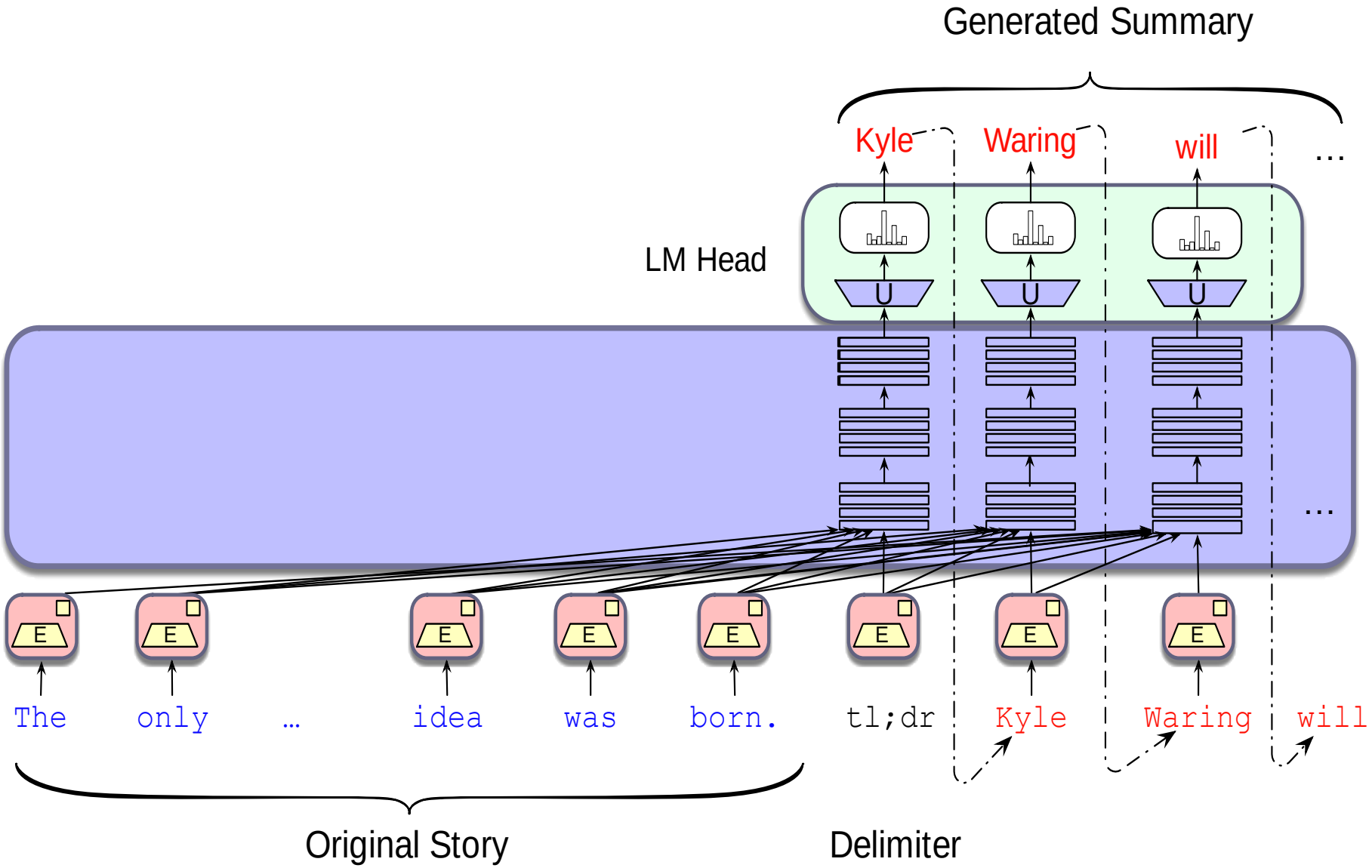
Summary

Kyle Waring will ship you 6 pounds of Boston-area snow in an insulated Styrofoam box – enough for 10 to 15 snowballs, he says. But not if you live in New England or surrounding states.

LLMs for summarization (using tl;dr)

- Give a LLM a text, and follow it by a token like **`tl;dr;`**
- Token **`tl;dr;`** is short for "*too long; didn't read*" often used, especially in informal work emails, when they are going to give a short summary.
- This token being sufficiently frequent in LM training data, the LM has seen many texts in which the token occurs before a summary, and hence will interpret it as instructions to generate a summary.
- So conditional generation ➔ give the LM the prefix, and then have it generate the following words, one by one.
- Take the entire response as a summary.

LLMs for summarization (using tl;dr)



Left-to-Right Generation

- The prediction of the next word is done from left to right for languages like English (right to left for Arabic).
- This is also called:
  - Causal generation

  (alternatives like the masked LMs are non-causal because they can predict words based on both past and future words).
  - Autoregressive generation
  - Left-to-right generation

Sampling for LLM Generation

Greedy Decoding

- One simple way to generate words is to always generate *the most likely word* given the context.
- This is called **greedy decoding (GD)**.
- A greedy algorithm is one that makes a locally optimal choice.

$$\hat{w}_t = argmax_{w \in V} P(w|w_{<t})$$

- In practice, greedy decoding not used with LLMs.
- Major problem with **GD**: because the words it chooses are extremely predictable, the resulting text is generic and often quite repetitive.
- GD will always result in generating exactly the same string.
- **Beam Search,** extension to GD, works well in tasks like MT.
- Most other tasks, more sophisticated methods, called sampling methods, introduce more diversity into the generations.

Decoding and Sampling

- Core of generation process for LLMs: task of choosing the word to generate next based on the context and the probabilities that the model assigns to possible words.
- This task is called **decoding**.
- **Sampling:** the most common method for decoding in LLMs.
- Sampling from a model's distribution over words: choose random words according to their probability assigned by the transformer model.
- After each token, sample words to generate according to their probability *conditioned on previous choices*.

Random sampling

- Algorithm for generating a sequence of words until the end-of-sequence token is met:

(x ~ p(x) means ‘choose x by sampling from the distribution p(x))

$$\mathrm{i} \leftarrow 1$$

$$w_i \sim \mathrm{p}(w)$$

while w_i != EOS

$$\mathrm{i} \leftarrow \mathrm{i} + 1$$

$$w_i \sim \mathrm{p}(w_i \mid w_{<i})$$

Random sampling Shortcoming

+ Random sampling mostly generates sensible, high-probable words,

- Problem: There are many odd, low-probability words in the tail of the distribution

  - Each one has low-probability but added up they constitute a large portion of the distribution.
  - So they get picked enough to generate weird sentences.

Factors in word sampling: quality and diversity

- Emphasize **high-probability** words
  - more **quality**: more accurate, coherent, and factual,
  - less **diversity**: boring, repetitive.
- Emphasize **middle-probability** words
  - more **diversity**: more creative, diverse,
  - less **quality**: less factual, incoherent

Top-k sampling:

- **Top-k sampling** is a simple generalization of greedy decoding.

1. Choose # of words k
2. For each word in the vocabulary V , use the LM to compute the likelihood of this word given the context $p(w_t \mid w_{<t})$
3. Sort the words by likelihood and keep only the top k most probable words.
4. Renormalize the scores of the k words to be a legitimate probability distribution.
5. Randomly sample a word from within these remaining k most-probable words according to its probability.

Top-p sampling (or nucleus sampling)

- Problem with top-*k* sampling: *k* is fixed so may cover very different amounts of probability mass in different situations.
- Idea: Instead, keep the **top p percent** of the probability mass.
- Given a distribution $P(w_t \mid \mathbf{w}_{<t})$, the top-*p* vocabulary $V(p)$ is the smallest set of words such that

$$\sum_{w \in V^{(p)}} P(w \mid \mathbf{w}_{<t}) \geq p$$

Temperature sampling

- Reshape the distribution instead of truncating it.
- Intuition from thermodynamics,
  - a system at high temperature is flexible and can explore many possible states;
  - a system at lower temperature is likely to explore a subset of lower energy (better) states.
- In **low-temperature sampling**, $(\tau \leq 1)$ we smoothly
  - increase the probability of the most probable words
  - decrease the probability of the rare words.

Temperature sampling

- Divide the logit by a temperature parameter τ before passing it through the softmax.

- Instead of

$$\mathbf{y} = \text{softmax}(u)$$

- We do

$$\mathbf{y} = \text{softmax}(u/\tau)$$

Temperature sampling

$$\mathbf{y} = \text{softmax}(u/\tau) \qquad 0 \leq \tau \leq 1$$

Why does this work?

- When τ is close to 1 the distribution doesn't change much.
- The lower τ is, the larger the scores being passed to the softmax function.
- Softmax pushes high values toward 1 and low values toward 0.
- Large input pushes high-probability words higher and low probability word lower, making the distribution more greedy.
- As τ approaches 0, the probability of most likely word approaches 1 .

Selection of τ

- In practice,
  - **temperature τ is typically a single fixed value**
  - applied uniformly across all logits for a given sample, and
  - its value is usually set via **parameter tuning** based on the desired level of diversity and creativity in the generated text.

Pretraining LLMs: Algorithm

Pretraining

- The big idea that underlies all the amazing performance of language models
  - First **pretrain** a transformer model on enormous amounts of text
  - Then **apply** it to new tasks.

Self-supervised training algorithm

- Model just trained to predict the next word!

1. Take a corpus of text
2. At each time step *t*
   i. ask the model to predict the next word ;
   ii. train the model using gradient descent to minimize the error in this prediction.

- Model called **self-supervised** because no need to add any special gold labels to the data; the natural sequence of words is its own supervision!
- Model trained to minimize the error in predicting the true next word in the training sequence, using **cross-entropy** as the loss function.

Cross-entropy loss for language modeling

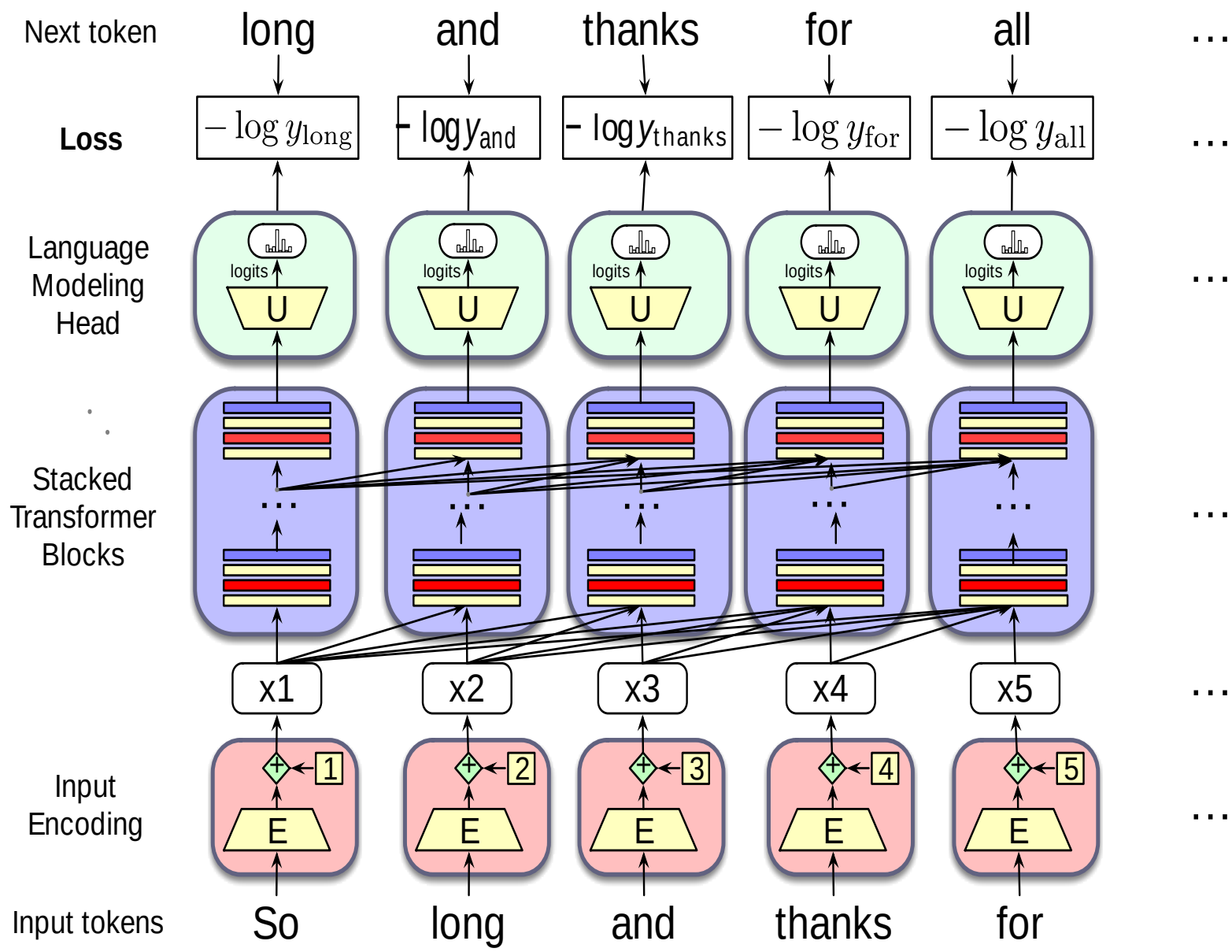
- **CE loss**: difference between the correct probability distribution and the predicted distribution

$$L_{CE} = -\sum_{w \in V} \mathbf{y}_t[w] \log \hat{\mathbf{y}}_t[w]$$

- The correct distribution $\mathbf{y}_t$ knows the next word, so it is 1 for the actual next word and 0 for the others.
- Hence in this sum, all terms get multiplied by zero except one: the log probability the model assigns to the correct next word, so:

$$L_{CE}(\hat{\mathbf{y}}_t, \mathbf{y}_t) = -\log \hat{\mathbf{y}}_t[w_{t+1}]$$

Training a transformer language model



$$= \frac{1}{T}\sum_{t=1}^{T} L_{CE}$$

Teacher forcing

- At each token position *t*, model sees correct tokens $w_{1:t}$,
  - Computes loss (–log probability) for the next token w_{t+1}
- At next token position t+1 we ignore what the model predicted for w_{t+1}
  - Instead we take the **correct** word w_{t+1}, add it to context, move on.
- For training, the probability assigned to the correct word is used to calculate the CE loss for each item in the sequence.
- The loss for a training sequence is the average cross-entropy loss over the entire sequence.
- Weights in the network are adjusted to minimize the average CE loss over the training sequence via gradient descent.

On LLM Training

- Large models are generally trained by filling the full context window with text.
  - For example, 4096 tokens for GPT4 or 8192 for Llama 3.
- If documents are shorter than this, multiple documents are packed into the window with a special end-of-text token between them.
- The batch size for gradient descent is usually quite large:
  - the largest GPT-3 model uses a batch size of 3.2 million tokens.

Parallelising & distributing the Computation

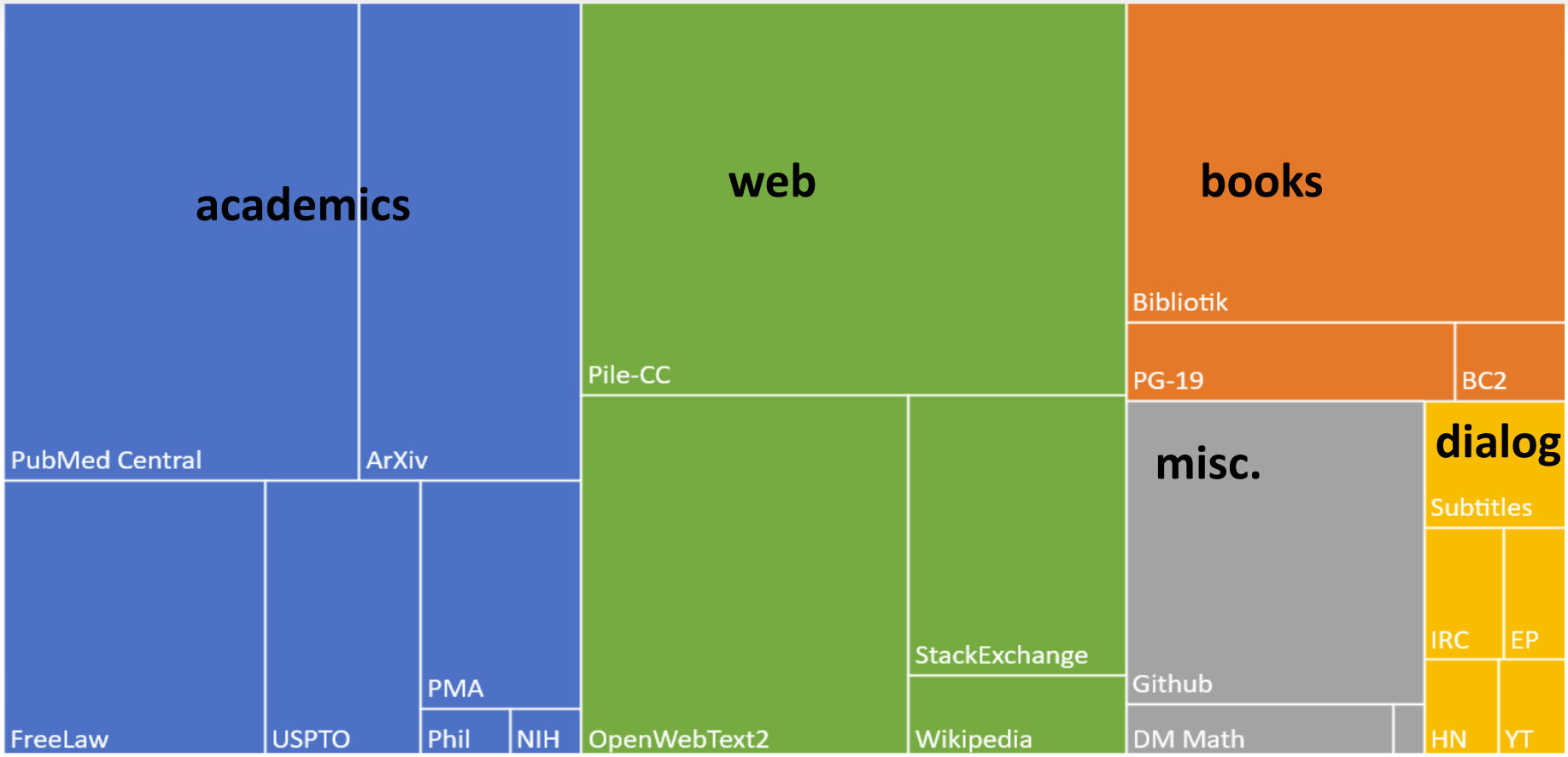
- **Time-sharing** is used so as multiple sequences are processed simultaneously, with resources (like GPUs/TPUs) taking turns processing different parts of the batch in parallel.
- The **parallelism** exists both **within sequences** (intra-sequence, e.g., attention heads) and **across sequences** (inter-sequence, e.g., parallel batches of sequences processed across GPUs/TPUs).
- **Parallelism** and **distributed processing** are what makes training on such large datasets and models possible.

Pretraining data for LLMs

LLMs are mainly trained on the web

- LLMs are mainly trained on
  - text scraped from the web,
  - augmented by more carefully curated data.
- Web text is usually taken from corpora of automatically-crawled web pages like
  - Common crawl, snapshots of the entire web produced by the non- profit Common Crawl with billions of pages.
- Various versions of Common crawl exist, e.g. Colossal Clean Crawled Corpus (C4),
  - 156 billion tokens of English, filtered. (deduplicated, removing non-natural language like code, sentences with offensive words from a blocklist).
- What's in it?
  - Mostly patent text docs, Wikipedia, and news sites

The Pile: a pretraining corpus



Dolma: a larger open corpus of English, created with public tools.
Three trillion tokens of web text, academic papers, code, books, encyclopedic materials, and social media.

Filtering for Quality and Safety

- **Quality** is subjective!
  - Quality filters are classifiers that assign scores to each doc.
    - Need to remove unwanted material such as Personal Identifiable Information (PII) and adult content.
    - Deduplication at many levels (URLs, documents, even lines)
  - Quality filtering usually improve LM performance.
- **Safety** is also subjective.
  - Toxicity detection is important, although that has mixed results.
  - Can mistakenly flag data written in dialects like African American English
  - LMs trained on toxicity-filtered content fail to detect toxicity.

Problems with scraping from the web

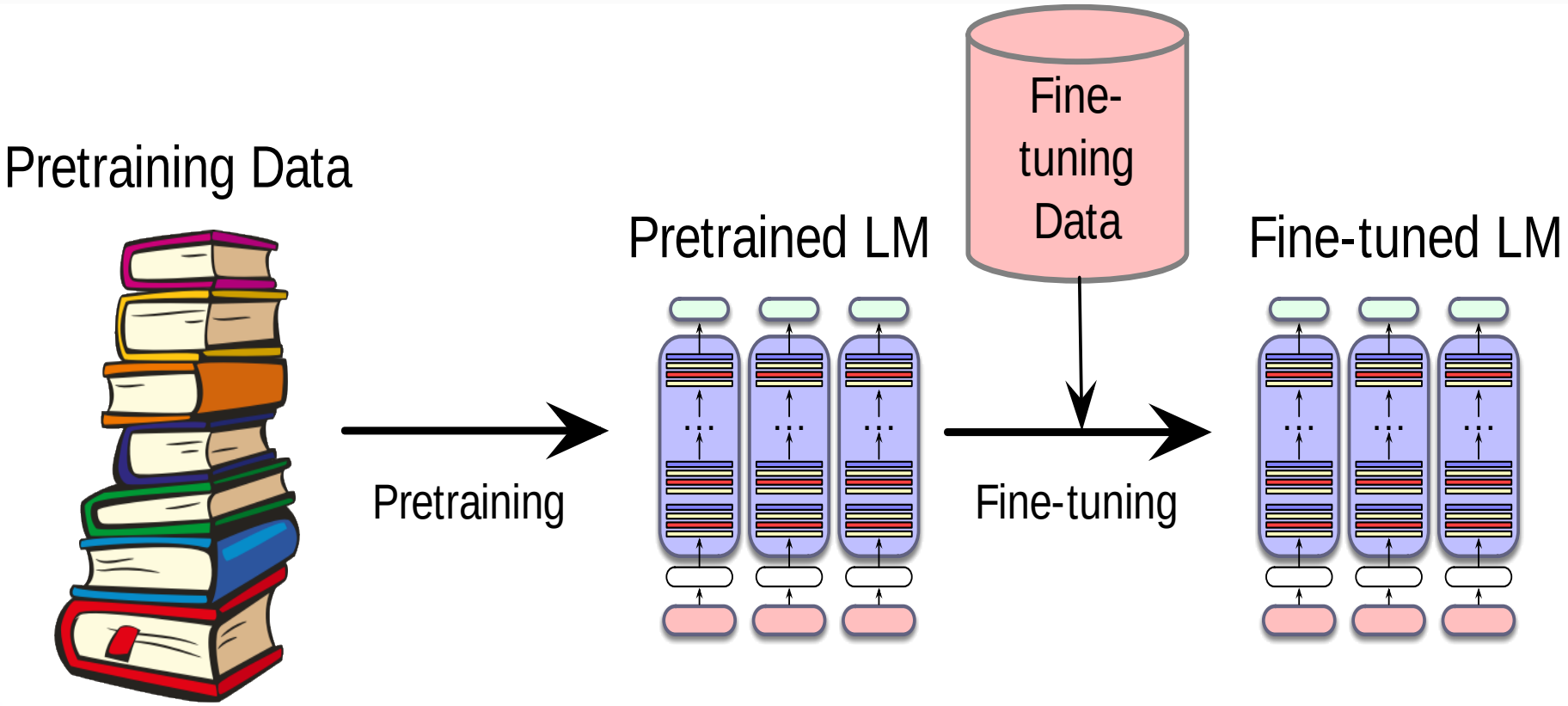
- **Copyright**: much of the text in these datasets is copyrighted (e.g. fiction and non-fiction books)
  - Not clear if fair use doctrine in US allows for this use. (copyrighted content to be used for transformative uses.)
  - Open legal question: LMs used to generate text competing with the market for text they are trained on.
- **Data consent**
  - Website owners can indicate they don't want their site crawled. (Recently, increase in such websites.)
- **Privacy**:
  - Websites can contain private IP addresses and phones.
  - Filters usually try to filter sites with lots of private data.

Finetuning

Finetuning

- What happens if we need our LLM to work well on a domain it didn't see in pretraining?
- Perhaps some specific medical or legal domain or task?
- Or maybe a multilingual LM needs to see more data on some language that was rare in pretraining?
- We can simply continue training the model on relevant data from the new domain or language.
- **Finetuning**: process of taking a fully pretrained model and running additional training passes on some new data.

Finetuning



- We will cover **4 types of "Finetuning"**, 2 in this chapter, and 2 in the next two chapters.
- Often everything that happens after pretraining is lumped together as **post-training.**

1. Finetuning as "continued pretraining" on new data

- Further train all the parameters of the model on new data
  - using the same method (word prediction) and loss function (cross-entropy loss) as for pretraining,
  - as if the new data were at the tail end of the pretraining data.
- Hence sometimes called **continued pretraining.**

Accessing the parameters of pretrained LM

- To fine-tune a model like **GPT-3** or **BERT**, need to access the **pre-trained model weights**.
- **Hugging Face Transformers Library**: one of the most common and convenient ways to access pre-trained models like BERT, GPT-2, …
- Hugging Face provides APIs that allow one to load pre-trained models directly and then fine-tune them on new data.
- The library takes care of handling the model's weights and the fine-tuning process.

e.g. for BERT

```
from transformers import BertForSequenceClassification, BertTokenizer
model_name = "bert-base-uncased" # Pre-trained BERT model
model = BertForSequenceClassification.from_pretrained(model_name)
Tokenizer = BertTokenizer.from_pretrained(model_name)
```

- This gives access to the **model's parameters** (weights) that can be fine-tuned.
- The new data having been processed, loaded and split, one can have:

```
# Fine-tune the model
model.fit(train_dataset_tf, validation_data =
                              eval_dataset_tf, epochs=epochs)
# Save the fine-tuned model & the tokenizer's configuration
model.save_pretrained("./fine_tuned_model")
tokenizer.save_pretrained("path_to_save_tokenizer")
```

Evaluating Large Language Models

Perplexity

- Just as for n-gram grammars, we use perplexity to measure how well the LM predicts unseen text
- The perplexity of a model θ on an unseen test set is the **inverse probability that θ assigns to the test set, normalized by the test set length.**
- For a test set of *n* tokens $w_{1:n}$ the perplexity is :

$$\text{Perplexity}_\theta(w_{1:n}) = P_\theta(w_{1:n})^{-\frac{1}{n}}$$

$$= \sqrt[n]{\frac{1}{P_\theta(w_{1:n})}}$$

Perplexity

- The higher the probability of the word sequence, the lower the perplexity.
- Thus the lower the perplexity of a model on the data, the better the model.
- **Minimizing perplexity is the same as maximizing probability**
- Also: perplexity is sensitive to length / tokenization so best used when comparing LMs that use the same tokenizer.

Other factors for evaluation

- Perplexity, is a very useful metric, but other kinds of accuracy metrics, for the downstream tasks (e.g. MT, summarization, QA, speech recognition, and dialogue) we apply our language model to will be presented.
- **Other factors for evaluation:**
  - **Size:** Big models take lots of GPUs and time to train, memory to store.
  - **Energy usage:** Can measure kWh or kilograms of CO2 emitted.
  - **Fairness:** Benchmarks measure gendered and racial stereotypes, or decreased performance for language from or about some groups.

Dealing with Scale

Scaling Laws

- LLM performance depends mainly on 3 factors:
  1. **Model size**: the number of parameters not counting embeddings
     - Llama 3.1 405B Instruct model from Meta: 405 billion parameters (126 layers, embedding dimension of 16,384, 128 attention heads) and trained on over 15 trillion tokens using a vocabulary of 128K tokens.
     - GPT-4o: estimated by many at 1.76 trillion parameters, context window 128K tokens, max output 16,384 tokens.
     - GPT-3: 175 billion parameters; embedding dimension of 12,288.

Scaling Laws

2. **Dataset size**: the amount of training data
   - Llama 3.1 405B trained on over 15 trillion tokens.
   - GPT-4 trained on 13 trillion tokens.

3. **Compute**: amount of compute (e.g. in FLOPS): typically in the order of **exaflops** during training (1 exaflop = 10^{18} FLOPS).

- A model can be improved by adding parameters (more layers, wider contexts), more data, or training for more iterations.
- The performance of a large language model (the loss) scales as a power-law with each of these three.

Scaling Laws

- Loss *L* as a function of the number of non-embedding parameters *N*, dataset size *D*, compute budget *C* (if the other two are held constant).

$$L(N) = \left(\frac{N_c}{N}\right)^{\alpha_N}$$

$$L(D) = \left(\frac{D_c}{D}\right)^{\alpha_D}$$

$$L(C) = \left(\frac{C_c}{C}\right)^{\alpha_C}$$

- The values of Nc, Dc, Cc, α_N, α_D and α_C depend on the exact transformer architecture, tokenization, and vocabulary size.
- For the initial experiment in Kaplan et al. (2020) $\alpha_N = 0.076$, Nc = $8.8\ 10^{13}$ (parameters), $\alpha_D = 0.095$, Dc = $5.4\ 10^{13}$ (tokens), $\alpha_C = 0.050$, Cc = $3.1\ 10^{8}$ (petaflop-days).
- A **petaflop-day** represents the total number of floating-point operations performed by a system that can compute **1 petaflop** (10^{15} operations per second) over the course of **one day** (86,400 seconds).

Estimating Number of Non-Embedding parameters N

- N can be roughly computed as follows (ignoring biases):

$$
\begin{aligned}
N &\approx 2\, d\, n_{\text{layer}}(2\, d_{\text{attn}} + d_{\text{ff}}) \\
&\approx 12\, n_{\text{layer}}\, d^2 \\
&\quad (\text{assuming } d_{\text{attn}} = d_{\text{ff}}/4 = d)
\end{aligned}
$$

where **d**: input and output dimensionality of the model,

d_{attn} : self-attention layer size,

d_{ff} : size of the feedforward layer.

- Thus GPT-3, with $n = 96$ layers and dimensionality $d = 12288$, has $12 \times 96 \times 12288^2 \approx 175$ billion parameters.
- Scaling laws can be used early in training to predict what the loss would be if we were to add more data or increase model size.

Parameter-Efficient Finetuning

- Adapting to a new domain by continued pretraining (finetuning) is a problem with huge LLMs.
  - Enormous numbers of parameters to train
  - Each pass of batch gradient descent has to backpropagate through many many huge layers.
  - Expensive in processing power, in memory, and in time.
- Instead, **parameter-efficient fine tuning** (PEFT)
- Efficiently select a subset of parameters to update when finetuning. E.g.
  - freeze some of the parameters (don't change them),
  - and only update some particular subset of parameters.

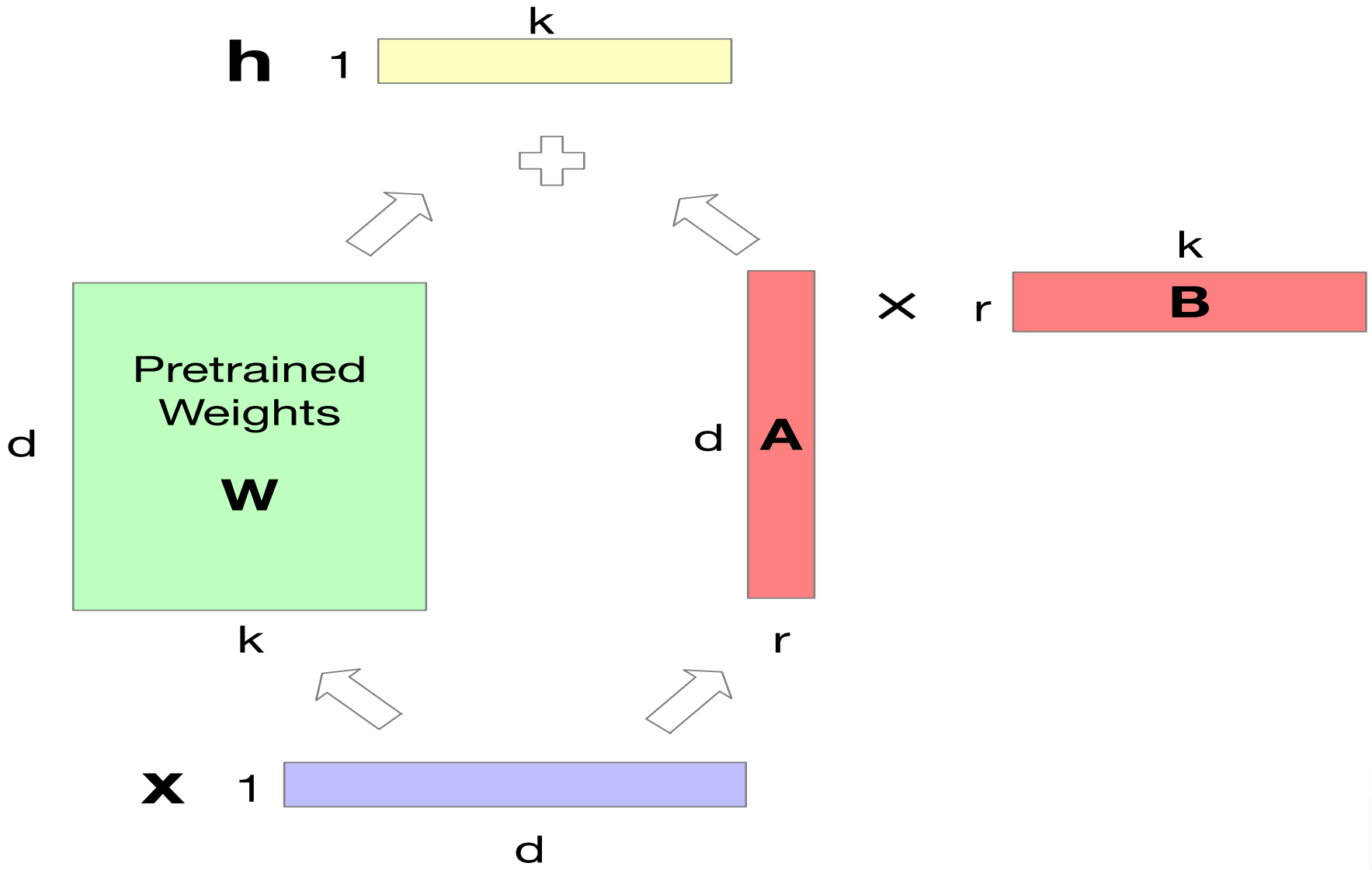
LoRA (Low-Rank Adaptation)

- **Intuition**: Transformers have many dense matrix layers that are used in multiplications
  - Like $\mathbf{W}^{\mathbf{Q}}$, $\mathbf{W}^{\mathbf{K}}$, $\mathbf{W}^{\mathbf{V}}$, $\mathbf{W}^{\mathbf{O}}$ layers in attention
- Instead of updating these layers during finetuning,
  - Freeze these layers
  - Update a low-rank approximation with fewer parameters.
- So LoRA provides an approach to adapting large models while significantly reducing the computational cost and memory footprint.

LoRA

- Consider a **matrix W** (shape $[d \times d_k]$) that needs to be updated during finetuning via gradient descent.
  - Normally, after Gradient Descent, W gets updates ΔW (of shape $[d \times d_k]$)
- In LoRA, W is frozen and, instead, a low-rank decomposition of W is updated (through training):
  - A of shape $[d \times r]$,
  - B of shape $[r \times d_k]$, *where* r very small (e.g. 1, 2, 4, 8, 16)
  - So, during finetuning, we update A and B instead of W.
  - Replace $W + \Delta W$ with $W + B\ A$.
- In forward pass, instead of $\mathbf{h = x\ W}$, we do $\mathbf{h = x\ W + x\ A\ B}$
- So LoRA introduces low-rank matrices that are trained, while the original pre-trained weight matrices remain frozen.

LoRA



Example

- Suppose $d_{model} = 512$, $d_k = 64$, $d_v = 64$, and Rank r = 8
- Original matrices:
  - W_Q: (512,64)
  - W_K: (512,64)
  - W_V:(512,64)
- For **queries**, the low-rank adaptation matrices would be:
  - A_Q: (512 , 8)
  - B_Q: (8, 64)
- In LoRA, for the *query*, instead of learning the entire W_Q (of size 512 x 64, i.e. 32,768 parameters), we only need to update the smaller matrices A_Q and B_Q , with a total of $512 \times 8 + 8 \times 64 = 4{,}096 + 512 = 4{,}608$ parameters.
- The same goes for W_K and W_V

Harms of Large Language Models

Hallucination



Chatbots May 'Hallucinate' More Often Than Many Realize

What Can You Do When A.I. Lies About You?

People have little protection or recourse when the technology creates and spreads falsehoods about them.

Air Canada loses court case after its chatbot hallucinated fake policies to a customer

The airline argued that the chatbot itself was liable. The court disagreed.

- Back to this issue in Chapter 14 where we introduce proposed mitigation methods like Retrieval Augmented Generation.

Toxicity and Abuse

The New AI-Powered Bing Is Threatening Users.

Cleaning Up ChatGPT Takes Heavy Toll on Human Workers

Contractors in Kenya say they were traumatized by effort to screen out descriptions of violence and sexual abuse during run-up to OpenAI's hit chatbot

- Toxic language even with completely non-toxic prompts.
- Generation of stereotypes and negative attitudes about various demographic groups.
- LLM developers are much more careful about their models.

Bias

- Training data disproportionately generated by authors from the US and from developed countries.
- Language models can amplify demographic and other biases in training data.
- Gender bias (based on the available data sets).
- Bias against ethnic groups.

Privacy



- LLMs can leak information from their training data ➔ an adversary can extract training-data text from a LM such as a person's name, phone number, and address
- Even more serious a problem when the LLM is trained on extremely sensitive private datasets such as electronic health records.

Copyright

Authors Sue OpenAI Claiming Mass Copyright Infringement of Hundreds of Thousands of Novels



The Times Sues OpenAI and Microsoft Over A.I. Use of Copyrighted Work

Millions of articles from The New York Times were used to train chatbots that now compete with it, the lawsuit said.



Feb 13, 2024 (Reuters) – "A federal judge in California has dismissed parts of a copyright lawsuit brought by comedian Sarah Silverman, Michael Chabon, Ta-Nehisi Coates and other authors against OpenAI over its alleged use of their books to train the large language model underlying its popular chatbot ChatGPT… OpenAI, Microsoft and other companies have said that their AI training is protected by the copyright doctrine of fair use and that the lawsuits threaten the burgeoning AI industry."

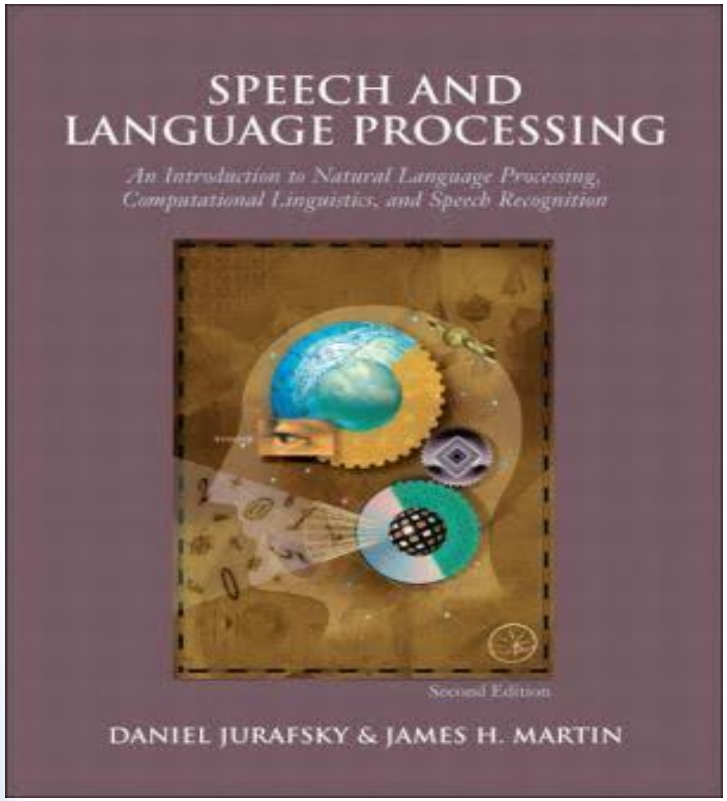
Misinformation

Chatbots are generating false and misleading information about U.S. elections

- LLMs can be used by malicious actors for generating text for
  - misinformation,
  - phishing, or
  - other socially harmful activities (extremist propaganda, etc.)

Textbook

Jurafsky, D. and Martin, J.H. (2024) *Speech and Language Processing: An Introduction to Natural Language Processing, Computational Linguistics, and Speech Recognition with Language Models* (3rd ed. draft), Prentice Hall.
https://web.stanford.edu/~jurafsky/slp3/ed3bookaug20_2024.pdf



The draft version of the 3rd edition will be used; it has been updated to include more recent topics such as Vector Semantics and Embeddings, ANNs and Deep Learning for NLP, Transformers, etc.

This course **slides** are largely based on this textbook and its authors' slides with light customisations whenever seen fit.